Part 1.1

For this part, the students are asked to reall all 10 images and resize them to 100 by 100. I am using a loop to read and save the images in a list. It is OK if they do it differently as long as the images are loaded and resized.

```
from PIL import Image
from pylab import *
```

```
## Part 1
images = []
for i in range(1, 11):
    name =  str(i) + '.PNG'
    im = Image.open(name)
    im_rs = im.resize((100, 100))
    images.append(im_rs)
```

```
# this is not required, we are just doing it to test that images are correctly loaded & processed.
imshow(images[0])
```

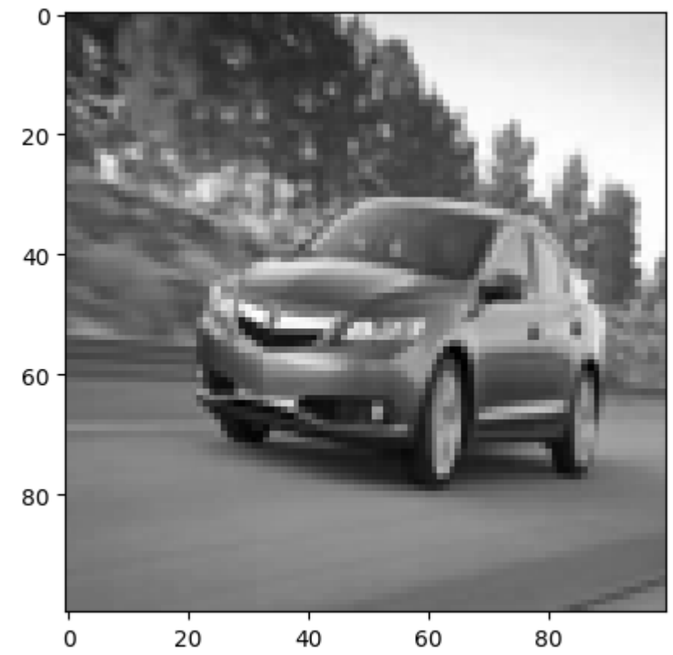
```
<matplotlib.image.AxesImage at 0x789d5a440110>
```

Part 1.2

This is straightforward. We are using a loop to convert all 10 images to grayscale using .convert('L) as discussed in the class. we are using imshow() to print a sample image, but that is not required to receive the points.

```
gray_images = []

for i in images:
    im_gray = i.convert('L')
    gray_images.append(im_gray)

imshow(gray_images[0], cmap = 'gray')
```

```
<matplotlib.image.AxesImage at 0x789d5a462de0>
```



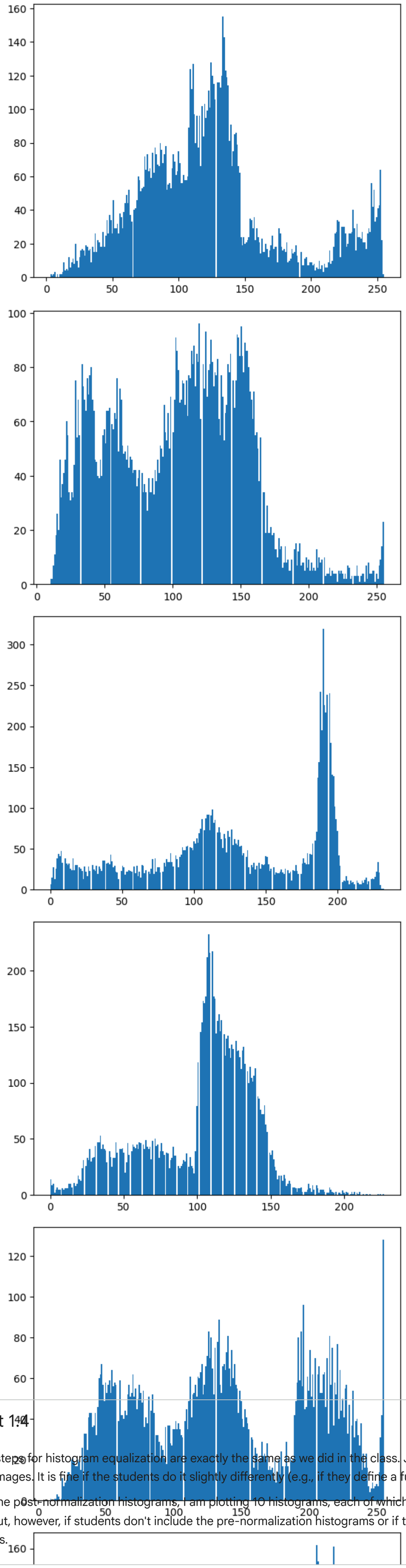
Part 1.3

We are using a loop to flatten each image. Then use another loop to create 10 histograms. Some students may have drawn all the histograms overlaid within the same plot. While, that is not ideal since it is very hard to see and distinguish them, we won't penalize them since the assignment description was not explicit about this.

```
import pandas as pd

flattened_images = []

for i in gray_images:
    im_v = array(i).flatten()
    flattened_images.append(im_v)

# These two lines are not required, I'm doing it for a sanity check and for demonstration.
print(flattened_images[0])
pd.DataFrame(flattened_images).to_csv("Flattened_Images.csv", index=False, header=False)
```

```
[ 42  37  36 ... 135 134 134]
```

```
for i in range(10):
    figure()
    hist(flattened_images[i], 256)
```



Part 1.4

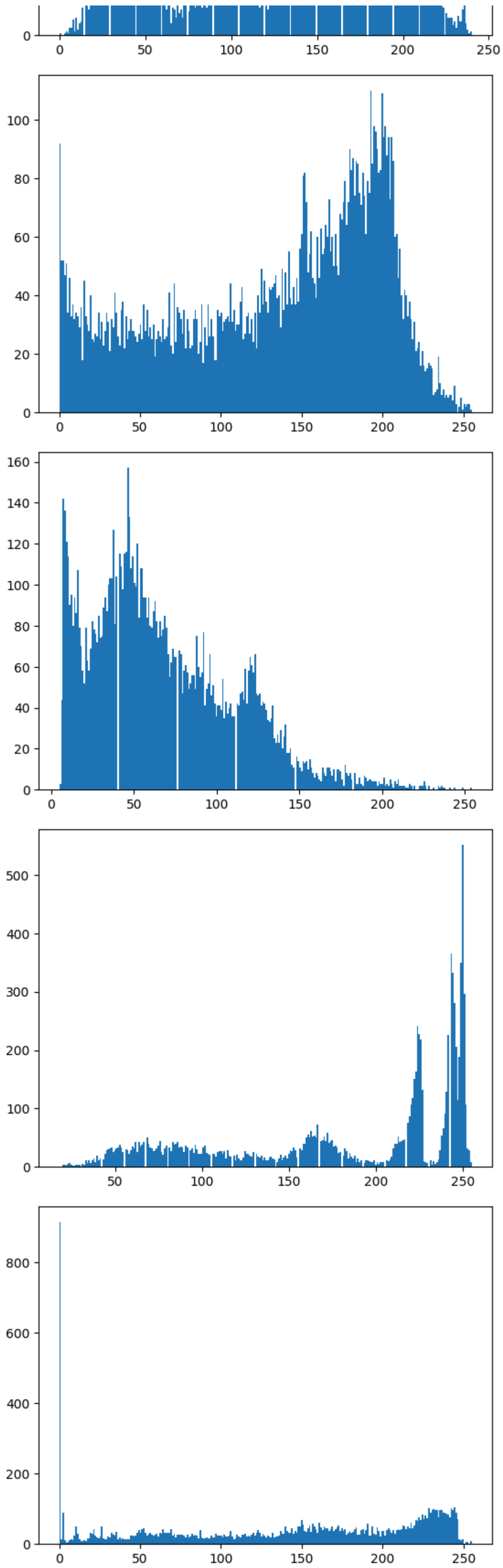
The steps for histogram equalization are exactly the same as we did in the class. Just that in this case, we need a loop to apply this to all the images. It is fine if the students do it slightly differently (e.g., if they define a function and apply the function to images, etc.)

For the post-normalization histograms, I am plotting 10 histograms, each of which has the pre-and post histograms. This is the ideal output, however, if students don't include the pre-normalization histograms or if they plot them all in the same plot, they still get the points.

```
equalized_images = []

for i in flattened_images:
    imhist, bins = histogram(i, 256, density=True)
    cdf = imhist.cumsum()
```

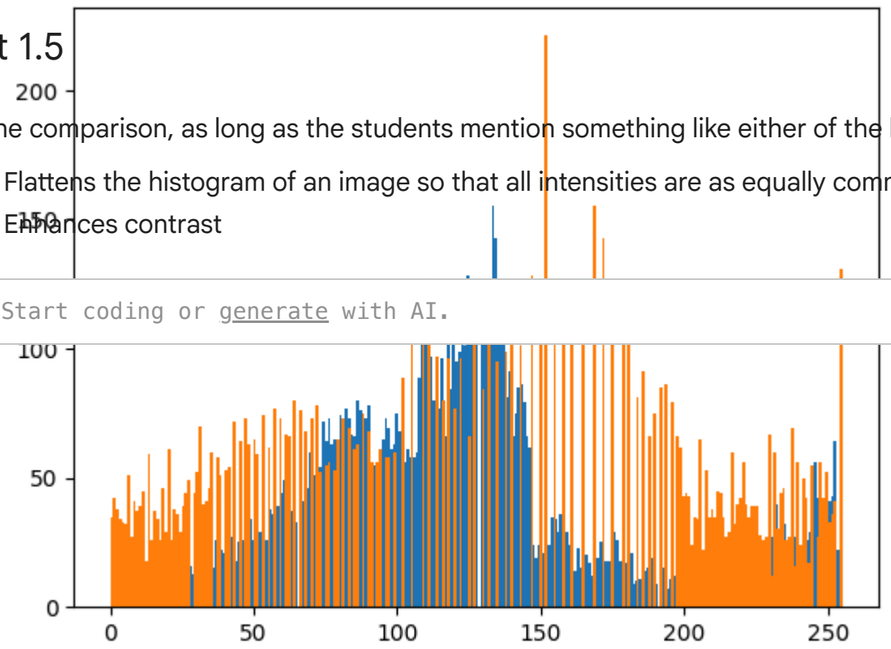
```
    cdf = 255 * cdf / cdf[-1]
    f_im = interp(i, bins[:-1], cdf)
    equalized_images.append(f_im)
```

```
for i in range(10): #they don't have to have the pre-normalized histogram in each plot
    figure()
    hist(flattened_images[i], 256)
    hist(equalized_images[i], 256)
```



Part 1.5

For the comparison, as long as the students mention something like either of the below, they get the points:

- Flattens the histogram of an image so that all intensities are as equally common as possible
- Enhances contrast

```
Start coding or generate with AI.
```